helixqa_integration_sec09

9. Phase 8: Enterprise UX Validation

Phase 8 validates that every client surface—Web, Desktop, Mobile, CLI, and TUI—delivers an enterprise-grade user experience for translation workflows. The scope covers interaction patterns, provider-model coverage across forty-two LLM providers, and the non-functional requirements (multi-tenancy, reliability, cost control, auditability) that separate a prototype from a production system. Findings are derived from the HelixCode client-application architecture, the HelixQA provider registry, and the `.env.example` configuration manifest[1].

9.1 Translation Tool UX Matrix

The HelixCode ecosystem ships five client applications: a browser-based Web interface, a Fyne v2 Desktop application, gomobile-based Android and iOS binaries, a Cobra/Viper CLI, and a `tview`/`tcell` Terminal User Interface (TUI). HelixQA must verify every translation-related interaction pattern with surface-appropriate tooling.

9.1.1 Web UX

The Web client is served by the Gin HTTP server and rendered in Chromium via Playwright. Translation features live on `/settings` (language selection), `/subtitles` (subtitle management), and `/media` (non-Latin filenames). The language-selector dropdown triggers a UI re-render plus an asynchronous `PUT /api/v1/config` call. Provider selection is surfaced through a configuration panel listing auto-discovered providers from `pkg/llm/providers_registry.go`. Model quality is communicated via a score badge from the LLMsVerifier registry (`VerifiedModel.Score`, refreshed every 300 seconds)[1].

9.1.2 Desktop UX

The Desktop application (`HelixCode/applications/desktop/`) uses Fyne v2 and compiles to native binaries for Windows, macOS, and Linux. It integrates through a system-tray icon (systray), native file dialogs, and global menu shortcuts. Language settings are presented in a modal dialog that writes to the platform-specific preferences store. Offline mode is supported when `HELIX_OLLAMA_URL` points to a local host; HelixQA validates this by disconnecting the test machine and confirming that `ollama` (`llava:7b` or `minicpm-v:8b`) still responds within 30 seconds.

9.1.3 Mobile UX

Android and iOS clients are produced via gomobile. The Android package uses Material3 components; the test bank (`banks/full-qa-android.yaml`) contains cases for Cyrillic search (`FQA-AND-044`) and biometric-dialog subtitle verification (`FQA-AND-012`). Touch targets must be at least 48 dp × 48 dp. Push notifications for long translations are surfaced via `internal/notification/`; HelixQA verifies delivery by triggering a long task and asserting that the notification payload contains the completed locale identifier.

9.1.4 CLI UX

The CLI (`HelixCode/cmd/cli/`, built with Cobra) exposes translation commands. Batch mode is exercised through `helix translate --batch --provider auto --format json`. Progress bars render as a spinner on `stderr`; HelixQA captures `stderr` and validates 10–60 updates per minute. Fallback chains are triggered by injecting a bad primary API key and asserting success through the secondary. Output formatting is validated for `json`, `csv`, and `txt` by schema-checking emitted bytes.

9.1.5 TUI UX

The TUI (`HelixCode/applications/terminal_ui/`, built with `rivo/tview`) is the richest keyboard-driven interface. Provider selection uses a list widget populated from the verifier cache; real-time streaming display is implemented through a `TextView` with `SetDynamicColors(true)` receiving WebSocket messages from `/ws/v1/chat`. Keyboard shortcuts (`Ctrl+P` provider panel, `Ctrl+M` model selection, `Ctrl+R` report export) are verified by sending `tcell` key events through the headless test harness. Color-coded quality scores use a four-tier palette: green ≥ 0.85, yellow 0.60–0.84, red < 0.60, gray for unverified providers.

Table 1. Complete UX Matrix: Client Type × UX Element × Interaction Pattern × QA Verification Method

Client	UX Element	Interaction Pattern	QA Verification Method	Pass Criteria
Web	Language selector dropdown	`<select>` change → `PUT /api/v1/config` → UI re-render	Playwright screenshot before/after + OCR diff	Labels differ; layout SSIM ≥ 0.95[1]
Web	Real-time preview	Screenshot → vision provider → text extraction	Anthropic/Gemini vision analysis of localized UI	Text matches target locale; no truncation
Web	Provider selection	Config panel lists auto-discovered providers	API contract: `GET /api/v1/llm/providers`	Response contains ≥ 1 provider with non-empty `Name`
Web	Model quality indicator	Score badge from LLMsVerifier	Schema validation on `VerifiedModel.Score`	Score ∈ [0.0, 1.0]; `Verified` boolean present
Desktop	Native OS integration	Menu shortcuts (`Ctrl+,` Settings, `Ctrl+Q` Quit)	AT-SPI2 (Linux), AXTree (macOS), UIA (Windows)	Shortcut triggers correct window state change
Desktop				

Client	UX Element	Interaction Pattern	QA Verification Method	Pass Criteria
	Tray icon status	Systray menu with provider health indicator	Window capture + pixel color assertion	Green dot when ≥ 1 provider healthy; red when all down
Desktop	Offline mode	No cloud keys + local Ollama URL	Disconnect network; run `helix chat --prompt "test"`	Response from `ollama` within 30 s
Mobile	Touch-optimized targets	Material3 buttons ≥ 48 dp	Appium `getSize()` assertion	Width ≥ 48, Height ≥ 48 in device-independent pixels
Mobile	Cyrillic on-screen keyboard	System IME switches to Cyrillic layout	ADB `input text` with Serbian Unicode + screenshot	Characters render without tofu (□) glyphs
Mobile	Push notifications	FCM/APNs payload for completed translation	Trigger long task; capture notification payload	Payload contains `locale` field matching request
CLI	Batch translation mode	`--batch` flag reads file list from stdin	Script 100 files; measure wall-clock time	All files processed; exit code 0; output valid JSON
CLI	Progress bars	Spinner on `stderr` during long operations	Capture `stderr` bytes; count spinner frames	10–60 updates/min; no interleaving with stdout
CLI	Provider fallback chains	Bad primary key → auto-fallback to secondary	Inject invalid `ANTHROPIC_API_KEY`; request succeeds	Request succeeds via `OPENAI_API_KEY` within timeout
CLI	Output formatting	`--format json/csv/txt`	Schema validate each format	JSON parses; CSV has header + equal columns; TXT is UTF-8
TUI	Interactive provider selection	`List` widget with verifier-populated items	Headless `tcell` key events → screenshot	Selected provider highlighted; detail pane updated
TUI	Real-time streaming display	`TextView` receives WebSocket chunks	Mock WebSocket server; inject tokens; capture screen	Text appends without flicker; ANSI colors stripped
TUI	Keyboard shortcuts	`Ctrl+P`, `Ctrl+M`, `Ctrl+R` mapped to actions	Send `tcell.EventKey` sequences; assert view change	Correct view gains focus within 100 ms

Client	UX Element	Interaction Pattern	QA Verification Method	Pass Criteria
TUI	Color-coded quality scores	Four-tier palette in provider list	Screenshot → parse foreground color codes	Green ≥ 0.85; yellow 0.60–0.84; red < 0.60; gray unverified

Table 1 maps nineteen UX elements across five client surfaces to concrete QA methods with measurable pass criteria. Web and Desktop rely on screenshot-plus-OCR pipelines (Tesseract for deterministic text extraction, LLM vision for semantic layout validation), Mobile requires Appium plus ADB instrumentation, and CLI/TUI lean on byte-capture and headless terminal emulation. This heterogeneity means HelixQA must maintain five separate driver backends—Playwright/chromedp, AT-SPI2/AXTree/UIA, Appium/UiAutomator2, Cobra command capture, and tcell—yet all routes converge on the same evidence format: screenshot or capture file plus structured JSON rationale.

9.2 Provider & Model Coverage

HelixCode's LLM subsystem (`HelixCode/internal/llm/`) implements a unified `Provider` interface with `Chat`, `Vision`, and `SupportsVision` methods. Auto-discovery at startup scans environment variables and registers every provider whose key is present. The `.env.example` file lists forty-two distinct provider environment variables, divided into Tier 1 (native adapters) and Tier 2 (OpenAI-compatible adapters)[1].

9.2.1 Tier 1 Providers (Native)

Tier 1 providers have bespoke Go adapter files (`*_provider.go`) in `HelixCode/internal/llm/`. Each adapter handles provider-specific authentication, request marshaling, response unmarshaling, and error-code translation. The fourteen native providers, their environment variables, default models, and vision capability are listed below.

Provider	Env Variable	Default Model	Vision
Anthropic	`ANTHROPIC_API_KEY`	`claude-sonnet-4`	Yes
OpenAI	`OPENAI_API_KEY`	`gpt-4o`	Yes
Google Gemini	`GEMINI_API_KEY`	`gemini-pro`	Yes
OpenRouter	`OPENROUTER_API_KEY`	`anthropic/claude-sonnet-4`	Varies
DeepSeek	`DEEPSEEK_API_KEY`	`deepseek-chat`	No
Groq	`GROQ_API_KEY`	`llama-3.3-70b-versatile`	No
Ollama	`HELIX_OLLAMA_URL`	`llava:7b` (local)	Optional
Astica	`ASTICA_API_KEY`	(specialized vision)	Yes
Kimi (Moonshot)	`KIMI_API_KEY`	`kimi-k2.5`	Yes
Qwen (Alibaba)	`QWEN_API_KEY`	`qwen-vl-max`	Yes
Stepfun	`STEPFUN_API_KEY`	`step-1.5v-mini`	Yes

Provider	Env Variable	Default Model	Vision
NVIDIA NIM	`NVIDIA_API_KEY`	`meta/llama-3.2-90b-vision-instruct`	Yes
xAI (Grok)	`XAI_API_KEY`	`grok-3`	Yes
Mistral	`MISTRAL_API_KEY`	`mistral-large-latest`	No

9.2.2 Tier 2 Providers (OpenAI-Compatible)

Tier 2 providers reuse a generic OpenAI-compatible adapter. The `factory.go` instantiates the same HTTP client with a configurable base URL and model name, which is why the system can absorb new providers without code changes when they follow the `chat/completions` contract.

Provider	Env Variable	Vision
AI21	`AI21_API_KEY`	No
Cerebras	`CEREBRAS_API_KEY`	No
Chutes	`CHUTES_API_KEY`	No
Cloudflare	`CLOUDFLARE_API_KEY`	No
Codestral	`CODESTRAL_API_KEY`	No
Cohere	`COHERE_API_KEY`	No
Fireworks	`FIREWORKS_API_KEY`	Yes
GitHub Models	`GITHUB_MODELS_API_KEY`	Yes
HuggingFace	`HUGGINGFACE_API_KEY`	Varies
Hyperbolic	`HYPERBOLIC_API_KEY`	Yes
Modal	`MODAL_API_KEY`	Varies
Novita	`NOVITA_API_KEY`	No
Perplexity	`PERPLEXITY_API_KEY`	No
PublicAI	`PUBLICAI_API_KEY`	No
Replicate	`REPLICATE_API_KEY`	Varies
SambaNova	`SAMBANOVA_API_KEY`	No
Sarvam	`SARVAM_API_KEY`	No
SiliconFlow	`SILICONFLOW_API_KEY`	No
Together AI	`TOGETHER_API_KEY`	Yes
Upstage	`UPSTAGE_API_KEY`	No
Venice	`VENICE_API_KEY`	No
Vulavula	`VULAVULA_API_KEY`	No
ZAI (BigModel)	`ZAI_API_KEY`	No
Zen	`ZEN_API_KEY`	No

Provider	Env Variable	Vision
Zhipu	`ZHIPU_API_KEY`	No
Junie	`JUNIE_API_KEY`	No
NLPCloud	`NLPCLOUD_API_KEY`	No
NIA	`NIA_API_KEY`	No

9.2.3 Vision-Capable Providers

Twelve providers expose vision endpoints: OpenAI (`gpt-4o`), Anthropic (Claude), Google Gemini, OpenRouter, Fireworks, Together AI, Hyperbolic, NVIDIA NIM, xAI Grok, Kimi K2.5, Qwen-VL, Stepfun Step-GUI, Astica (specialized caption/OCR), and Ollama (when `llava`, `minicpm-v`, `bakllava`, or `moondream` is pulled). Vision provider selection uses the scoring formula

$$\text{Score} = (0.6 \times \text{QualityScore} + 0.4 \times \text{ReliabilityScore}) \times \text{AvailabilityBoost} \times \text{CostBonus}$$

where `AvailabilityBoost` is 1.0 if the API key is configured and 0.5 otherwise, while `CostBonus` is 1.10 for free providers (Ollama), 1.05 for providers costing less than $0.002 per 1k tokens, and 1.0 otherwise[1]. The ranked output drives adaptive routing: a screenshot containing Cyrillic text is routed to Anthropic or Gemini (highest OCR accuracy for non-Latin scripts), while a routine UI-navigation screenshot may be routed to Kimi K2.5 ($0.0006/1k tokens, 0.88 quality) to minimize burn rate.

9.2.4 QA Verification: Every Provider, Every Script

HelixQA must verify that every configured provider returns a real translation (not an empty string or generic error placeholder) and that every vision-capable model handles Cyrillic and Unicode correctly.

Table 2. Provider Coverage Matrix: Tier, Vision, Translation Verified, Fallback Tested, Cyrillic Tested

#	Provider	Tier	Vision	Translation Verified	Fallback Tested	Cyrillic Tested
1	Anthropic	1	Yes	Mandatory	Mandatory	Mandatory
2	OpenAI	1	Yes	Mandatory	Mandatory	Mandatory
3	Google Gemini	1	Yes	Mandatory	Mandatory	Mandatory
4	OpenRouter	1	Varies	Mandatory	Mandatory	Mandatory
5	DeepSeek	1	No	Mandatory	Mandatory	Mandatory
6	Groq	1	No	Mandatory	Mandatory	Mandatory
7	Ollama	1	Optional	Mandatory	Mandatory	Mandatory
8	Astica	1	Yes	Mandatory	Mandatory	Mandatory
9	Kimi	1	Yes	Mandatory	Mandatory	Mandatory
10	Qwen	1	Yes	Mandatory	Mandatory	Mandatory

#	Provider	Tier	Vision	Translation Verified	Fallback Tested	Cyrillic Tested
11	Stepfun	1	Yes	Mandatory	Mandatory	Mandatory
12	NVIDIA	1	Yes	Mandatory	Mandatory	Mandatory
13	xAI	1	Yes	Mandatory	Mandatory	Mandatory
14	Mistral	1	No	Mandatory	Mandatory	Mandatory
15	AI21	2	No	Mandatory	N/A	Mandatory
16	Cerebras	2	No	Mandatory	N/A	Mandatory
17	Chutes	2	No	Mandatory	N/A	Mandatory
18	Cloudflare	2	No	Mandatory	N/A	Mandatory
19	Codestral	2	No	Mandatory	N/A	Mandatory
20	Cohere	2	No	Mandatory	N/A	Mandatory
21	Fireworks	2	Yes	Mandatory	N/A	Mandatory
22	GitHub Models	2	Yes	Mandatory	N/A	Mandatory
23	HuggingFace	2	Varies	Mandatory	N/A	Mandatory
24	Hyperbolic	2	Yes	Mandatory	N/A	Mandatory
25	Modal	2	Varies	Mandatory	N/A	Mandatory
26	Novita	2	No	Mandatory	N/A	Mandatory
27	Perplexity	2	No	Mandatory	N/A	Mandatory
28	PublicAI	2	No	Mandatory	N/A	Mandatory
29	Replicate	2	Varies	Mandatory	N/A	Mandatory
30	SambaNova	2	No	Mandatory	N/A	Mandatory
31	Sarvam	2	No	Mandatory	N/A	Mandatory
32	SiliconFlow	2	No	Mandatory	N/A	Mandatory
33	Together AI	2	Yes	Mandatory	N/A	Mandatory
34	Upstage	2	No	Mandatory	N/A	Mandatory
35	Venice	2	No	Mandatory	N/A	Mandatory
36	Vulavula	2	No	Mandatory	N/A	Mandatory
37	ZAI	2	No	Mandatory	N/A	Mandatory
38	Zen	2	No	Mandatory	N/A	Mandatory
39	Zhipu	2	No	Mandatory	N/A	Mandatory
40	Junie	2	No	Mandatory	N/A	Mandatory
41	NLPCloud	2	No	Mandatory	N/A	Mandatory
42	NIA	2	No	Mandatory	N/A	Mandatory

Table 2 contains forty-two rows, one per provider identified in `.env.example` and the provider registry[1]. The "Fallback Tested" column is marked "N/A" for Tier 2 because fallback chains are implemented at the Tier 1 adapter level (`adaptive.go`); Tier 2 providers are invoked through the generic OpenAI-compatible adapter, which itself falls back to another Tier 1 provider when the requested Tier 2 endpoint returns a 5xx or 429 status. Every provider is marked "Mandatory" for Cyrillic testing because the system guarantees Unicode fidelity across the entire surface; this is enforced by `T-I18N-010` in the test bank[1].

Specific Test Cases for Translation Workflow Validation

HelixQA executes three canonical test cases during every regression run that covers i18n functionality.

Test Case TC-TRANS-01: Cyrillic Search and Display (Serbian "Титаник")

Field	Value
Objective	Verify that Cyrillic query strings are preserved end-to-end through search input, API transport, database storage, and result rendering.
Input	Search query `Титаник` (Serbian Cyrillic for "Titanic").
Steps	1. Navigate to `/media`. 2. Enter `Титаник` in the search field. 3. Submit. 4. Capture screenshot of results. 5. OCR with Tesseract (script map `"Cyrillic": "rus"`). 6. Verify API response contains `"query": "Титаник"`. 7. Export results to JSON/CSV/TXT and verify encoding.
Provider Routing	Vision: Anthropic Claude (primary) or Google Gemini (secondary). API validation: Groq (fastest) or DeepSeek (cheapest). Fallback: Ollama `minicpm-v:8b` if all cloud keys fail.
Pass Criteria	OCR extracts `"Титаник"` with Character Error Rate (CER) = 0. API payload preserves exact Unicode code points. Export files are valid UTF-8; no `U+FFFD` replacement characters. Layout SSIM ≥ 0.95 compared to Latin-query baseline.

Test Case TC-TRANS-02: Language Setting Persistence Across Sessions

Field	Value
Objective	Confirm that a user's language preference survives logout, login, and application restart.
Input	Language code `sr` (Serbian).
Steps	1. Authenticate. 2. Navigate to Settings → Language. 3. Select `sr`. 4. Capture

Field	Value
	screenshot; OCR verify localized labels. 5. Logout. 6. Clear browser cookies (Web) or wipe `NSUserDefaults` key (Desktop) or force-stop app (Mobile). 7. Re-authenticate. 8. Capture screenshot of landing page. 9. OCR verify still localized. 10. Query `GET /api/v1/config`; assert `language` field.
Provider Routing	API validation: Groq. Vision validation: Anthropic.
Pass Criteria	Landing page text is Serbian after re-authentication without re-selecting language. API returns `"language": "sr"`. No regression to default locale (`en`).

Test Case TC-TRANS-03: Subtitle Download Encoding Integrity

Field	Value
Objective	Verify that subtitle files downloaded through the API retain UTF-8 encoding and render Cyrillic characters correctly in external players.
Input	Media ID with Serbian subtitle track; download endpoint `GET /api/v1/subtitles/download/{id}?lang=sr`.
Steps	1. Query `GET /api/v1/subtitles/languages`; assert `sr` present. 2. Download `.srt` file. 3. Byte-level inspection: assert BOM absent, valid UTF-8 sequence. 4. Parse SRT; assert subtitle text blocks contain Cyrillic. 5. Open file in VLC (Desktop) or system player (Mobile); screenshot with subtitles enabled. 6. OCR screenshot; compare against ground-truth subtitle text.
Provider Routing	Vision: Anthropic (subtitle detail). API: DeepSeek (cost-sensitive bulk check).
Pass Criteria	File is valid UTF-8 without BOM. SRT index blocks are well-formed. OCR CER against ground truth = 0. Video screenshot shows Cyrillic overlay without truncation.

These three cases form the core Cyrillic/Unicode validation suite. TC-TRANS-01 exercises the critical path from user input to visual output; TC-TRANS-02 validates state durability, a frequent source of enterprise support tickets; TC-TRANS-03 ensures binary artifact integrity for subtitle-distribution workflows. All three are tagged `i18n`, `cyrillic`, and `subtitles` in the test-bank YAML and are filtered automatically when HelixQA runs with `--tags i18n,cyrillic`[1].

9.3 Enterprise Quality Requirements

Enterprise translation tooling demands guarantees beyond functional correctness. The HelixCode system addresses four non-functional domains that HelixQA must continuously validate.

9.3.1 Multi-Tenancy

Multi-tenancy is implemented through per-session isolation. Each HelixQA session receives a unique session identifier and an isolated browser profile (incognito mode, fresh `localStorage`, separate cookie jar). For desktop and mobile tests, the device pool assigns discrete emulator or physical-device slots. Per-tenant provider quotas are enforced in `pkg/llm/cost_tracker.go`: every `Chat` and `Vision` call records token counts and estimated cost in a per-session `llm_usage` JSON object. HelixQA validates isolation by running two concurrent sessions with different language settings and asserting that Session A's Cyrillic search history does not appear in Session B's `localStorage` after both complete.

9.3.2 Reliability

The reliability target is 99.9% uptime for the LLM routing layer, measured as the ratio of successful requests to total requests over a 30-day rolling window. The `adaptive.go` file implements circuit breakers with three cooldown tiers: 60 seconds for HTTP 429 (rate limit), 300 seconds for HTTP 5xx (server error), and 120 seconds for network timeout[1]. HelixQA validates reliability through a chaos test: every provider is blocked in turn (via `iptables` DROP or invalid key injection), and the system must complete a standard translation task without human intervention by following the fallback chain Vision: Anthropic → OpenAI → Ollama (`llava`) → FAIL, and Reasoning: Groq → Cerebras → DeepSeek → OpenAI → Anthropic → FAIL[1]. Automatic failover is confirmed when the session report lists the blocked provider in `unavailable_providers` and the successful provider in `actual_provider`.

9.3.3 Cost Control

Cost control operates at three layers. First, per-request billing is recorded in the `llm_usage` JSON attached to every session report. Second, budget caps are enforced by the `cost_tracker.go` middleware: when the running total exceeds a configurable threshold (default $10.00 per session), subsequent requests are routed to Ollama or rejected with error `ErrBudgetExceeded`. Third, provider cost optimization uses the `CostBonus` factor in the vision-ranking formula to prefer cheaper providers for low-risk tasks. HelixQA validates cost control by running the full i18n test suite and asserting that the summed `estimated_cost_usd` in the final report matches an independently calculated projection within ±5%. Caching is validated by repeating an identical Cyrillic search twice and asserting that the second invocation does not increment the provider token counter.

9.3.4 Auditability

Auditability is governed by Constitution §11.4, which mandates that every QA decision carries a non-empty `Rationale` field and at least one evidence artifact[1]. For translation workflows, the evidence set must include: (a) a screenshot of the localized UI, (b) the OCR-extracted text, (c) the API request/response payload, and (d) the provider routing decision with cost. Full request/response logging is implemented in `internal/logging/` with structured JSON output; sensitive fields are redacted via a `Sanitize` middleware. Screenshot evidence is stored in `HELIX_OUTPUT_DIR` with filenames following `{session_id}_{test_id}`

`_{timestamp}.png`. Compliance reporting aggregates these artifacts into a Markdown + HTML + JSON bundle suitable for Sarbanes-Oxley (SOX) or General Data Protection Regulation (GDPR) review. HelixQA validates auditability by inspecting the output directory after a translation test run and asserting that every `T-I18N-*` test case has at least four evidence files and a non-empty `Rationale` in the JSON report.